Module 1: Introduction to Version Control

Section 1: Course Introduction

- This course introduces the fundamental concepts and practical use of version control systems (VCS), focusing on Git and GitHub, to help learners manage code and configuration files effectively.

Version Control Systems and Their Importance

- VCS tools like Git allow tracking changes, rolling back mistakes, and collaborating efficiently on code and configuration files.
- Understanding version control is essential for IT roles, enabling better project execution and teamwork.

Git and GitHub Overview

- The course covers Git's core functionalities, including basic and advanced features like branching and merging.
- Learners will set up Git on their computers and create remote repositories on GitHub to store and share their work.

Course Structure and Learning Outcomes

- The course uses Python scripts as examples, recommending basic Python knowledge and familiarity with Linux command line.
- By the end, learners will be able to manage code history with Git, collaborate via GitHub, and build a portfolio useful for IT job interviews.

--Check **"READING_01"** file--

Section 2: Before Version Control

- This lecture introduces the importance of version control systems in managing IT scripts and configuration files, highlighting how they help track changes, facilitate collaboration, and enable safe rollbacks.

Challenges in IT Change Management

- IT environments involve managing evolving automation scripts and configuration files that change over time due to new features, security needs, and infrastructure growth.

- Keeping detailed historical records of changes is critical for troubleshooting, documentation, and undoing changes without relying on memory.

Benefits of Version Control Systems

- Version control systems allow administrators to track what was changed and when, providing a reliable way to revert to previous working versions if new changes cause issues.
- Using version control avoids the pitfalls of quick fixes that may introduce new bugs, enabling safer, tested code releases.

Introduction to Git and Course Overview

- The course will teach how to use Git, a popular version control system, to track changes in scripts and configuration files.
- Learners will install Git, understand its basic workflow, and eventually use GitHub for remote collaboration, with guidance provided throughout the process.

- This lecture introduces the concept of version control, explaining its purpose and benefits in managing project changes over time.

Understanding Primitive Version Control

- Primitive version control involves manually creating copies of work to preserve earlier versions and track progress.
- This method is limited because it requires remembering to make copies, often duplicates entire projects, and lacks detailed tracking of who made changes and why.

The Role of Version Control Systems

- Version control systems automate the tracking of changes in files such as code, images, or configurations.
- They help users compare different versions, understand the history of changes, and collaborate more effectively.

Introduction to Git

- Git is a popular version control system that helps manage changes and collaboration.
- The course will cover Git's core features, terminology, and practical uses to improve control over project history and teamwork.

--Check **"READING_02"** file--

- This lecture explains how to compare differences between two versions of code files using command-line tools.

Comparing Files with Diff

- The diff command shows only the lines that differ between two files, making it easier to spot changes than visually comparing files side by side.
- Symbols like "<" and ">" indicate lines removed from the first file and lines added to the second file, respectively.

Understanding Diff Output Formats

- Diff output can show changes as replaced lines, added lines, or multiple changes split into sections with line numbers and change types (e.g., "c" for changed, "a" for added).
- Using the "-u" flag with diff shows a unified format with context lines, using "-" for removed lines and "+" for added lines, clarifying complex changes like new code blocks.

Other Tools for Comparing Files

- Alternatives to diff include wdiff, which highlights word-level changes instead of line-level.
- Graphical tools like meld, KDiff3, and vimdiff provide side-by-side views with color highlighting to better visualize differences.

--Check **“READING_03”** file--

- This lecture explains how to use diff and patch commands to manage and apply changes in code files effectively.

Understanding Diff Files

- Diff files show the differences between an old file and a new file, including context to help understand the changes.
- They are generated using commands like diff -u old_file new_file > change.diff and are also called patch files.

Applying Changes with Patch

- The patch command automates applying changes from a diff file to the original file, avoiding manual edits.
- You use patch by redirecting the diff file as input, e.g., patch original_file < change.diff.

Practical Example and Benefits

- An example script checking CPU load is fixed by applying a diff file that adds a parameter and a debug line.

- Using diff and patch helps manage changes even if the original code has evolved and is especially useful for modifying multiple files in large projects.

--Check **“READING_04”** file--

- This lecture explains how to use diff and patch commands to identify and fix bugs in a Python script.

Fixing the Script

- The script disk_usage.py has bugs causing errors, such as a return statement outside a function.
- The fix involves replacing the return with sys.exit and correcting redundant unit conversions in the disk space calculation.

Using diff and patch

- Copies of the original and fixed scripts are made to compare changes.
- The diff command generates a file showing differences between the two versions.
- The patch command applies these changes to the original script to fix it.

Practical Application and Next Steps

- The fixed script runs successfully after applying the patch.
- This manual process demonstrates the usefulness of diff and patch before moving on to version control systems like Git.

--Check **“Study_Guide_01”** file--

Section 3: Version Control Systems

- This lecture explains the concept and importance of a Version Control System (VCS) in managing changes to files, especially in software development.

What is a Version Control System (VCS)

- A VCS tracks changes made to files, recording when and who made the changes.
- It allows reverting changes, merging edits from multiple sources, and treating a set of edits as a single commit with descriptive information.

How VCS Works and Its Benefits

- Unlike regular file servers, a VCS stores all versions of files, not just the latest.

- It helps understand complex changes, debug issues, and collaborate efficiently by managing repositories with multiple contributors.

Use Cases and File Types in VCS

- VCS is essential in organizations for managing code, configuration files, documentation, and other content.
- It works best with text files for diffing and patching but can also store complex files like images or videos, though merging and diffing are harder for these formats.

- This lecture explains the value of using a Version Control System (VCS) like Git, even for individual IT specialists, by highlighting how it helps manage code and configuration files with a history of changes.

Benefits of Using a VCS

- A VCS stores code and configuration files along with their change history, acting like a time machine to understand past decisions.
- Commit messages document why changes were made, helping avoid confusion when revisiting old code or configurations.

Practical Examples in IT

- Storing DNS zone files in a VCS allows tracking changes, quick rollback to previous versions, and improved system reliability.
- For DHCP servers with primary and standby machines, using a VCS ensures identical configuration files across machines, simplifying updates and troubleshooting.

Additional Advantages

- VCS audit trails speed up problem resolution by enabling quick rollbacks before debugging.
- Automating server deployment becomes easier with all configurations stored and versioned in a VCS.

- This lecture introduces Git, a popular version control system (VCS) created by Linus Torvalds in 2005 to manage the development of the Linux kernel.

Overview of Git

- Git is a free, open-source VCS available on Unix-based platforms, Windows, and macOS.
- It was designed to handle collaboration among geographically distributed developers efficiently.

Distributed Architecture

- Git uses a distributed model where every contributor has a full copy of the repository locally.
- This allows fast operations and flexible collaboration without relying on a centralized server.

Git Usage and Communication

- Git can be used standalone, as a server, or as a client, supporting offline work and networked collaboration.
- Communication between Git clients and servers can occur over HTTP, SSH, or Git's own protocol.

Flexibility and Popularity

- Git supports projects of all sizes, from single developers to thousands of contributors.
- It can manage private work or shared projects hosted on platforms like GitHub or GitLab.

Terminology and Alternatives

- Git is sometimes referred to as SCM (Source Control Management), which is synonymous with VCS.
- Other VCS tools like Subversion and Mercurial exist, but Git is chosen here for its popularity and robust features.

--Check **"More_Information_about_git"** file—

- This lecture focuses on installing Git and using it through the command line across different operating systems.

Installing Git

- Check if Git is already installed by running git --version; if the version is older than 2.20 or not found, installation is needed.
- Use package managers like apt, yum, Chocolatey, or Homebrew for easy installation, or download the installer from the official Git website.

Using Git on Different Operating Systems

- On Linux, install Git with simple commands like apt install git or yum install git.
- On Mac OS, running git --version may prompt automatic installation; otherwise, download and install manually.
- On Windows, the installer includes MinGW64, allowing Linux-like command line usage; default installation options are usually fine, but you may want to select a preferred text editor.

Working with Git Command Line and IDEs

- The course emphasizes using Git via the command line for standardization and flexibility.
- Graphical interfaces in IDEs are acceptable, but mastering the command line ensures proficiency with any Git tool.

- This lecture explains how to install Git on a Windows machine, covering the installation steps and configuration options.

Downloading and Initial Setup

- Download the latest Git installer from gitforwindows.org, which includes Git, a Bash emulation environment, Unix-like utilities, and a graphical interface.
- Accept the GPL version 2 license and choose the installation path (default is recommended).

Installation Options and Configuration

- Select additional components like Windows Explorer integration, large file support, and file associations for Git configuration and shell scripts.
- Choose your preferred text editor for Git (e.g., Atom, Visual Studio Code), noting that editors must be installed separately.
- Configure the PATH environment to allow Git commands in the embedded terminal and/or Windows Command Prompt.

Advanced Settings

- Decide on SSL certificate validation method (OpenSSL or Windows native).
- Configure line ending handling to ensure compatibility across different operating systems.
- Choose the terminal emulator (Git's own or Windows default) and enable performance and credential management options.
- Optionally enable experimental features, though the default is to keep them disabled for stability.

--Check **"Installing_git"** file--

Section 4: Using Git

--Check **"READING_05"** file--

- This lecture introduces the fundamental concepts and initial setup steps for using Git, a version control system.

Git Configuration and Initialization

- Git requires user identity configuration using `'git config'` to track who makes changes, with options for global or repository-specific settings.

- A Git repository can be created from scratch with `'git init'` or cloned from an existing remote repository using `'git clone'.`

Git Directory and Working Tree

- Running `'git init'` creates a hidden `.git` directory that acts as a database storing the project's change history.
- The working tree is the visible project directory where files are modified; initially, it may be empty.

Tracking and Committing Changes

- Files in the working tree start as untracked; using `'git add'` moves them to the staging area (index) to prepare for committing.
- The `'git commit'` command saves staged changes to the repository with a commit message, marking a new point in the project's history.

--Check **"READING_06"** file--

- This lecture explains the core concepts of how Git tracks changes in a project through three main areas: the Git directory, the working tree, and the staging area.

Git Project Structure

- The Git directory stores the history of all files and changes as snapshots.
- The working tree reflects the current state of the project files, including any modifications.

File Tracking and States

- Files can be tracked (part of snapshots) or untracked (new files not yet tracked).
- Tracked files go through three states: modified (changed but not staged), staged (marked for next commit), and committed (changes saved in a snapshot).

Typical Git Workflow

- Modify files in the working tree.
- Stage changes using the `'git add'` command to prepare them for commit.
- Commit staged changes with `'git commit'`, creating a new snapshot in the Git directory.
- Use `'git status'` to check the current state of files and get helpful command tips.

This explanation helps clarify how Git manages file changes and versions, emphasizing the cycle of modifying, staging, and committing files to maintain a project's history.

--Check **"READING_07"** file--

- This lecture explains the basic Git workflow and key concepts involved in using Git for version control.

Git Repository Setup and Configuration

- Initialize a new Git repository with git init in a directory to start tracking files.
- Check configuration with git config -l, paying attention to user.name and user.email for commit identity.

File States and Workflow

- Files can be untracked, modified, staged, or committed in Git.
- Use git add to move files from untracked or modified to staged, preparing them for commit.

Committing Changes

- Commit staged changes with git commit; this records a snapshot of the project.
- Commit messages can be entered via a text editor or directly with git commit -m.

Practical Example

- Created a Python script file, made it executable, and tracked it with Git.
- Modified the script by adding a function, staged the changes, and committed them with a descriptive message.

Overall, the workflow involves creating or modifying files, staging changes with git add, and committing them with git commit to save project history. Practice is encouraged to become comfortable with these commands.

--Check **“READING_08”** file--

- This lecture explains the importance of writing clear and informative commit messages in Git to help future developers or yourself understand changes made to the code.

Commit Message Structure

- A good commit message starts with a short summary line (about 50 characters), followed by a blank line.
- After the blank line, a detailed description explains why the changes were made and any important context, usually wrapped at 72 characters.

Best Practices for Commit Messages

- Keep the first line concise and descriptive, avoiding vague terms like "update" or "fix."
- Include references to related bugs, issues, or design documents to provide additional context.

Using Git Log and Commit Metadata

- The git log command shows commit history including unique commit IDs, author information, timestamps, and commit messages.
- Lines starting with a pound symbol (#) in the commit message editor are comments and not included in the commit.

Following these guidelines makes commit messages more useful and easier to understand when reviewing project history.

--Check **"Study_Guide_02"** file--

Section 5: Module 1 Review

This lecture summarizes the key concepts covered in the module about version control and Git, emphasizing practical application and preparation for assessments.

Key Concepts Covered

- Introduction to version control and its importance for collaboration and individual work.
- Basic Git concepts and the typical Git workflow for tracking changes.

Practical Application

- Encouragement to practice Git commands and workflows to solidify understanding.
- Use of Qwiklabs platform for hands-on labs simulating real-world Git scenarios.

Assessment Preparation

- The graded assessment involves initializing a repository, adding content, and committing changes.
- Emphasis on practicing these steps to build confidence for future modules.

--Check **"Glossary"** file--

--Check **"Qwiklabs_guidelines_&_troubleshooting_steps"** file--

--Check **"Exemplar"** file--